

Informe Técnico Final: Ten Floors



Integrantes:

- Aguilar Axel
- Chia Gianluca
- Salto Lautaro

Materia

Algoritmos y Estructura de Datos II

Narrativa del Dominio del Problema

El Dominio del Problema: Ecosistema MMORPG

El desarrollo de un videojuego de rol multijugador masivo en línea (MMORPG) como "Ten Floors" impone desafíos críticos en la persistencia en memoria, indexación y velocidad de procesamiento de datos. En un entorno de producción real, miles de entidades dinámicas (cuentas de usuario, inventarios locales, árboles de habilidades cambiantes, transacciones comerciales masivas y solicitudes de transporte global) interactúan de forma simultánea.

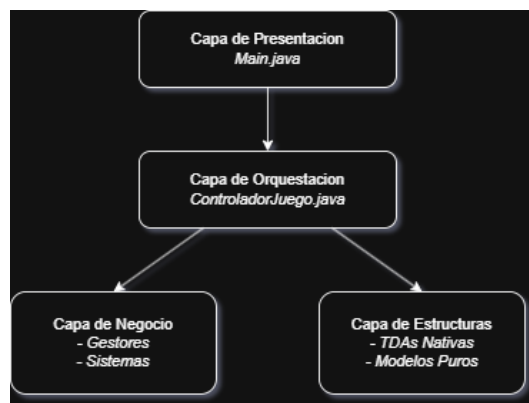
Este proyecto busca modelar el backend de dicho ecosistema. La arquitectura simula la progresión de los usuarios a lo largo de una torre colosal de 10 pisos únicos, regulando la jerarquía de gremios mediante relaciones de grafo y árboles de habilidades, garantizando que cada mutación del estado del juego responda a reglas de negocio.

Arquitectura General del Sistema Y Control de Flujos

Control de Flujo de Navegación Mediante el TDA Pila

El flujo de pantallas dentro de Main.java implementa un patrón de Máquina de Estados Finitos respaldado explícitamente por el TDA nativo Pila (historialMenus). Cuando el usuario avanza hacia un submenú (ej. de "PRINCIPAL" a "GESTION_DATOS"), el identificador del estado se añade a la cima mediante una operación apilar (push). Cuando selecciona la opción de regresar, se ejecuta desapilar (pop), lo que destruye el estado actual y redirige el menú anterior. El bucle principal (while) evalúa constantemente el tope de la pila (peek / verTope) para determinar la rutina de renderizado a invocar, garantizando una navegación robusta y libre de acoplamiento.

Desacoplamiento en Capas



El programa adopta una variante estricta del patrón Controlador-Mediador para segregar las responsabilidades del sistema. Se estructuran tres capas claramente diferenciadas:

- **Capa de Presentación (Main.java):** Es la capa externa y la única que posee interactividad con los flujos de entrada y salida. Su responsabilidad se limita a la captura de datos, conversión a mayúsculas para asegurar consistencia de IDs y delegación inmediata de la lógica de control. No almacena el estado del negocio.
- **Capa de Orquestación y Mediación (ControladorJuego.java):** Actúa como un orquestador. Centraliza las instancias de todos los TDAs globales y coordina las comunicaciones entre los gestores especializados. Oculta por completo la complejidad de las estructuras al Main; este último solicita operaciones de alto nivel (ej. darAltaCuenta o procesarDespachoMision), ignorando por completo si por detrás se ejecutan rotaciones AVL por ejemplo.

- **Capa de Lógica de Negocio y Estructuras de Datos (gestor/, consulta/, tda/):**
Almacena las reglas de negocio puras (como los requisitos de nivel para misiones y sincronización automática de habilidades) y encapsula el comportamiento interno de las estructuras implementadas nativamente.

Mecánica Detallada del Ciclo de Vida: Registro y Despacho de Misiones

Un excelente ejemplo de la interacción coordinada entre la arquitectura en capas y las estructuras de datos se evidencia en el método `procesarDespachoMision` del `ControladorJuego`:

1. **Validación del Índice Principal:** El sistema interroga al Árbol AVL (`indiceCuentas.buscar(idCuenta)`). Si el usuario no existe, aborta el ciclo de forma segura sin alterar el estado de la aplicación.
2. **Extracción por Prioridad:** Se extrae el elemento con mayor urgencia de la Cola de Prioridad administrada por el `GestorMisiones`.
3. **Evaluación de Reglas de Dominio:** Se extrae el `pisoActual` del modelo Jugador y se compara contra el `pisoRequerido` de la misión obtenida.
Si el jugador no cumple la condición, la misión se procesa pero se descarta por nivel insuficiente, protegiendo la integridad del progreso de la torre.
4. **Mutación e Inserción Cruzada:** Si pasa las pruebas, el identificador de la recompensa se busca en el catálogo maestro (ABB Global) y, al hallarse, se clona e inserta directamente en la mochila local del jugador (otro ABB Local), garantizando que las referencias de datos se mantengan estables.
5. **Evolución Jerárquica:** Finalmente, al incrementar el nivel del jugador, el `GestorHabilidades` ejecuta un algoritmo sobre el Árbol Genérico N-ario de habilidades globales, evaluando qué nuevos nodos de destrezas pasivas o activas deben anexarse al Conjunto de habilidades aprendidas del personaje de acuerdo a su nivel y clase.

Desglose de TDAs

Grafo No Dirigido (Mapa del Mundo)

Estructura Interna y Punteros

Implementado de forma nativa mediante una Lista de Adyacencia. Utiliza nodos de vértices (NodoVertice) que mantienen la información de la zona, una referencia al primer nodo de su lista de aristas (listaAdyacencia), y un puntero de encadenamiento lineal hacia el siguienteVertice de la torre. Las aristas (NodoArista) guardan un puntero directo al vértice destino y un enlace a la siguienteArista de la adyacencia.

Complejidad

- **agregarVertice(T info):** $O(V)$ en el peor caso, debido a que recorre la lista enlazada de vértices hasta el extremo final para insertar el nuevo nodo y asegurar que no existan zonas duplicadas. $O(1)$ por inserción.
- **agregarArista(T origen, T destino):** $O(V + E_v)$, donde primero localiza ambos vértices en la lista ($O(V)$) y luego itera sobre sus listas de adyacencia locales ($O(E_v)$) para evitar portales redundantes antes de enlazar al frente en $O(1)$.
- **Recorridos (bfs / dfs):** $O(V + E)$ al visitar exhaustivamente cada vértice y evaluar todas sus conexiones incidentes. La función bfs utiliza una estructura de cola auxiliar nativa (NodoColaAuxiliar) administrada de manera manual en memoria para controlar la exploración.

Árbol Genérico N-ario (Árbol de Habilidades)

Estructura Interna y Punteros

Estructurado mediante la representación de "Primer Hijo / Siguiendo Hermano" (primerHijo y siguienteHermano). Esta arquitectura elimina la rigidez de arreglos estáticos de hijos, permitiendo que cualquier nodo de habilidad o clase se ramifique dinámicamente sin límites.

Complejidad

- **agregarHijo(T datoPadre, T datoHijo):** $O(N + S)$, donde requiere una búsqueda en profundidad ($O(N)$) para localizar el nodo padre dentro del árbol de talentos, seguido de un recorrido lineal ($O(S)$) sobre la cadena de hermanos para anexar la nueva aptitud al final de la secuencia. $O(1)$ (adición de un solo nodo referenciado).
- **Recorridos (preorden / postorden):** $O(N)$, visitando cada nodo exactamente una vez para imprimir jerarquías tabuladas o auditar dependencias del juego. $O(H)$ en la pila de llamadas de ejecución recursiva, dependiente de la altura máxima (H) del árbol.

Árbol AVL (Índice Global de Usuarios)

Estructura Interna y Punteros

Árbol binario auto-balanceado cuyas claves de ordenación son de tipo String (ID de Cuenta). Cada NodoAVL almacena el ID de cuenta, el objeto genérico encapsulado, referencias a sus hijos izquierdo/derecho y un entero primitivo para computar la altura del subárbol.

Mecanismo de Balanceo

Durante las fases recursivas de insertar y eliminar, el método evalúa el factorEquilibrio ($FE = altura_{izq} - altura_{der}$). Si $FE > 1$, se mitiga la asimetría ejecutando rotaciones puras en punteros, garantizando estabilidad.

Complejidad

- **insertar, buscar y eliminar:** Temporal $O(\log V)$ estricto tanto para el caso promedio como para el peor escenario. Al rebalancearse constantemente, la altura queda acotada respecto a la cantidad de cuentas indexadas. $O(V)$ en almacenamiento total del índice.

ABB - Árbol Binario de Búsqueda (Inventario del Jugador)

Estructura Interna y Punteros

Árbol binario convencional estructurado mediante nodos dinámicos (NodoABB) que ordenan posicionalmente los ítems de la mochila del jugador utilizando su ID alfanumérico como clave de ramificación binaria (izquierdo menores, derecho mayores).

Complejidad

- **insertar, buscar y eliminar:** Promedio $O(\log N)$ bajo una distribución de inserción aleatoria. En el Peor Caso es $O(N)$ si los ítems se ingresan en orden estrictamente secuencial (provocando que el ABB de la mochila degenere en una lista enlazada asimétrica). El proceso de remoción contempla tres casos de eliminación (hoja, un hijo, o dos hijos sustituyendo mediante el sucesor mínimo del subárbol derecho).

Árbol B (Historial de la Casa de Subastas)

Estructura Interna y Punteros

Estructura balanceada multirrama diseñada para la persistencia masiva en memoria. Cada nodo (NodoB) cuenta con estructuras estáticas internas paralelas: un arreglo primitivo de claves numéricas `long[]` claves, un arreglo de objetos asociados `T[]` datos y un arreglo de referencias a nodos descendientes `NodoB<T>[]` hijos.

Algoritmo de División (Split)

Configurado de manera balanceada (grado mínimo parameterizado, inicializado en $t=2$ para pruebas globales en controlador). Cuando el método `insertar` detecta de forma preventiva que un nodo hijo ha alcanzado su capacidad máxima permitida de $(2t - 1)$ elementos, invoca `dividirHijo(x, i, y)`. Este método extrae la clave mediana, la promueve al nodo padre, y divide las claves y punteros restantes de manera equitativa distribuyéndolos hacia un nuevo nodo hermano `z`.

Complejidad

- **insertar y buscar:** $O(\log_t N)$ debido al alto factor de ramificación que reduce drásticamente la profundidad del árbol en transacciones masivas.

Pila (Historial de Menús / Comercio)

Estructura Interna

Estructura lineal encadenada dinámicamente con comportamiento estrictamente LIFO (Last In, First Out) mediante un puntero único de acceso denominado `tope`.

Complejidad

- **apilar(T elemento), desapilar() y verTope():** $O(1)$ garantizado de forma determinista, dado que toda mutación estructural o lectura ocurre directamente sobre el nodo apuntado en la cima, sin requerir iteraciones sobre la colección.

Cola Estándar (Espera de Mazmorras)

Estructura Interna

Estructura lineal FIFO (First In, First Out) que mantiene dos referencias de puntero explícitas en sus extremos de control:

- primero (para operaciones de salida/desencolado)
- último (para operaciones de inserción/encolado).

Complejidad

- **encolar(T elemento) y desencolar():** $O(1)$ estricto. Gracias a la referencia directa al puntero último, el encolamiento se realiza de forma inmediata sin recorrer la cola.

Cola con Prioridad (Registro de Misiones / Tickets VIP)

Estructura Interna

Implementada mediante una Lista Enlazada Simplemente Ordenada en base a un peso numérico entero (prioridad) de forma descendente.

Complejidad

- **insertar(T elemento, int prioridad):** $O(P)$ en el peor caso (donde P es el tamaño de la cola). Recorre linealmente los nodos buscando el punto exacto de inserción ordenada descendente para asegurar que las misiones o reclamaciones VIP críticas queden posicionadas inmediatamente al frente.
- **extraerMaximo():** $O(1)$ al remover directamente el nodo ubicado en la cabecera (primero), el cual posee la mayor urgencia computada.

Conjunto Basado en Tabla Hash (Jugadores Online)

Estructura Interna y Colisiones

Implementa un Set nativo estructurado sobre un arreglo dinámico de baldes (`Nodo<T>[]` tabla) administrando las colisiones mediante la estrategia clásica de Encadenamiento Separado (Linked List) en cada índice.

Redimensionamiento (Rehash)

Opera con una capacidad inicial de 16 baldes y un factor de carga crítico de 0.75. Cuando la proporción entre elementos almacenados y el tamaño del arreglo supera dicho umbral, se dispara el método privado `rehash()`, el cual duplica el tamaño del arreglo y redistribuye posicionalmente cada nodo.

Complejidad

- **agregar, contiene y eliminar:** Promedio $O(1)$ gracias a una dispersión uniforme y el control estricto del factor de carga. Peor caso $O(N)$ solo ocurre si todos los elementos terminan guardados en la misma posición por una mala distribución del hash, lo que obliga al sistema a recorrerlos uno por uno.

Resolución de Hitos Complejos

Hito 1: Viaje Rápido y Formación de Party (SistemaViajeYParty.java)

El componente recibe las zonas origen/destino y un arreglo lineal de personajes candidatos a conformar el equipo de exploración. Primero invoca el recorrido en anchura bfs sobre la instancia global del Grafo de adyacencia para trazar e imprimir las rutas viables y validar la conectividad física de la torre. Posteriormente, itera linealmente el arreglo de candidatos y somete cada entidad a una verificación inmediata contra el Conjunto hash de cuentas conectadas (`jugadoresOnline.contiene(jugador)`). Si el personaje está online, se añade secuencialmente a una Cola FIFO nativa que modela la party asignada a la mazmorra.

Hito 2: Soporte Técnico VIP (SistemaSoporteVIP.java)

Diseñado para la resolución y auditoría técnica de reclamaciones de usuarios prioritarios. El método extrae en primer término el ticket con mayor nivel de urgencia computada de la ColaPrioridad ejecutando `extraerMaximo()` en tiempo constante. Con el ID de transacción extraído del reclamo, el sistema ejecuta una búsqueda recursiva sobre el Árbol B histórico de subastas para validar la legitimidad de la transacción. Acto seguido, busca la entidad de cuenta en el Árbol AVL global para verificar la cuenta del reclamante. Finalmente, tras validar el ítem compensatorio en el catálogo maestro gestionado por un ABB, se inserta el ítem de compensación directamente en el árbol de inventario privado del jugador.

Hito 3: Auditoría de Gremios (SistemaAuditoriaGremios.java)

Ejecuta una verificación orgánica sobre el organigrama de un gremio para depurar y consolidar de forma única sus líderes en el servidor. El proceso inicia una rutina recursiva en profundidad sobre el Árbol Genérico N-ario del gremio utilizando la navegación por punteros de hijo y hermano. En cada nodo jerárquico se extrae el ID de cuenta, se interroga al índice Árbol AVL global de usuarios para corroborar su existencia real en el juego, y los perfiles de Jugador validados se insertan en un Conjunto hash temporal de líderes. El Set rechaza automáticamente cualquier inserción duplicada, logrando purgar líderes redundantes o recursivos.

Hito 4: Sistema de Comercio Seguro (SistemaComercioSeguro.java)

Permite la reversión de la última operación de un jugador para mitigar fraudes o transacciones erróneas. El flujo localiza la cuenta afectada mediante una búsqueda binaria por String ID dentro del Árbol AVL global. Una vez verificada la cuenta, se accede a su Pila de historial comercial local y se extrae el último objeto de intercambio aplicando `desapilar()`. Se extrae el ID del ítem involucrado en la transacción y se valida su integridad con el catálogo base de ítems del servidor implementado en un Árbol ABB global. Si el ítem es legítimo, se inserta de vuelta directamente en el ABB de inventario local de la mochila del

jugador. Como las estructuras comparten los mismos datos del jugador en memoria, cualquier cambio en la mochila se refleja automáticamente en el árbol AVL global sin necesidad de reinsertar la cuenta.